Vector Databases and RAG

How we search meaning,
not just words

We search wrong

Traditional search =
string matching

cats ≠ felines

We need meaning

Text → Vector

We convert text into numbers

Meaning → Geometry

Similar meaning =
close vectors

Find nearest neighbors

Can SQL
do this?

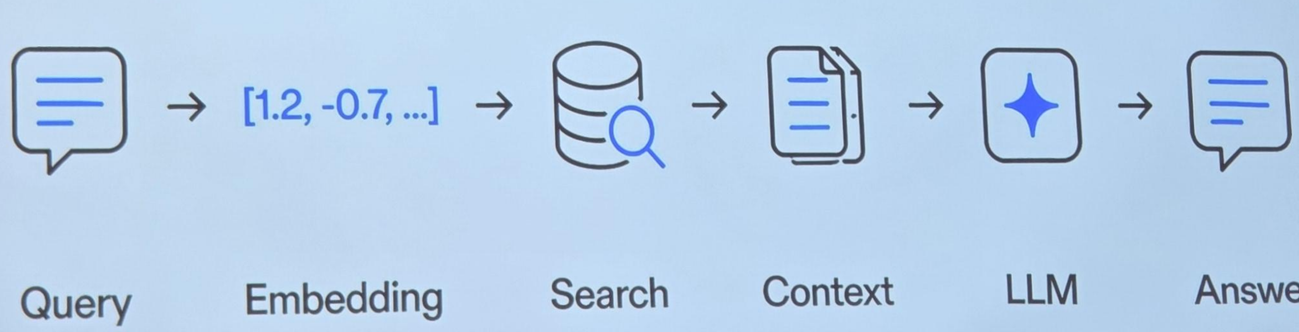
No index

B-tree
doesn’t work

ANN

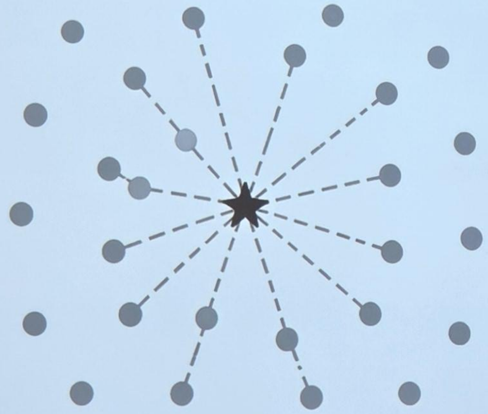
Approximate Nearest
Neighbor

RAG: how it works



ANN vs Brute Force

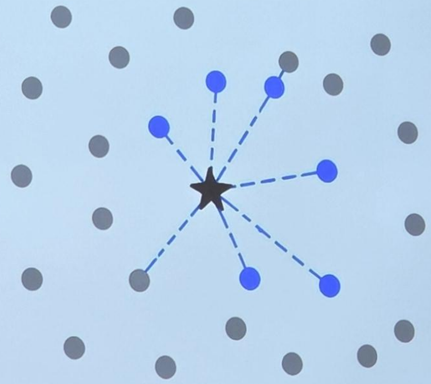
Brute Force (Exact)



Compare with all vectors

High accuracy, low speed

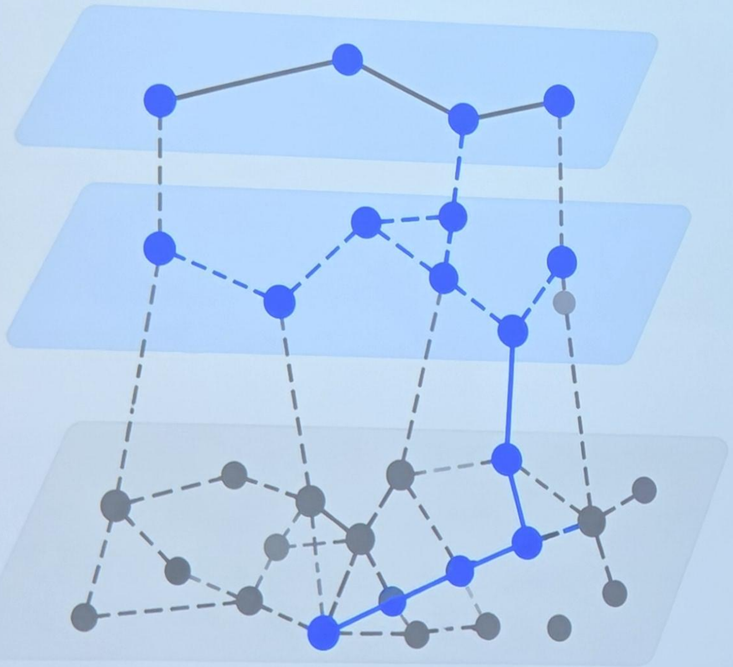
ANN (Approximate)



Compare with a subset

High speed, slightly less accurate

HNSW: Graph Search



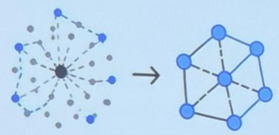
1. Start at the top layer
2. Move to nearest neighbors
3. Go down a layer
4. Repeat until bottom layer
5. Return best neighbors

Vector DB Systems

	FAISS	Chroma	Milvus	Pinecone
Type	Low-level library	Easy to use	Scalable	Managed cloud service
Performance	High performance	Open source	High performance	Fully managed
Use case	Customizable	Great for apps	Enterprise-ready	Reliable & secure

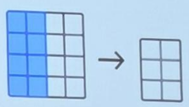
Vector DB Features

More than just nearest neighbor search



Index Types

- **HNSW** – graph-based, fast search
- **IVF** – partitions into clusters
- **PQ** – compress vectors for efficiency

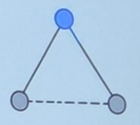


Compression

- Quantization (float → int)
- Product Quantization (PQ)
- Less memory, faster search

Metadata Filtering

- Filter by attributes
- Example: language = 'en' and date > 2023



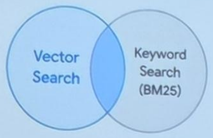
Distance Metrics

- Cosine Similarity
- Dot Product
- L2 (Euclidean)

Advanced Capabilities

Powerful features for real-world applications

Hybrid Search

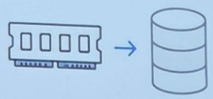


- Combine vector search with keyword search
- Better relevance and accuracy

Scalability

- Sharding & distribution
- Distributed ANN search
- Handle billions of vectors

Memory Management

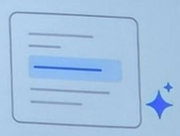


- Hot data in RAM, cold data on disk
- Cache & prefetch
- Optimal resource usage

Dynamic Updates

- Insert, update, delete vectors
- Different strategies: HNSW vs IVF

RAG Optimizations



- Store document chunks
- Return top-k with context
- LLM-friendly integrations

We search **meaning**

From keywords to understanding